Quantum Noise

Why Near-Term Quantum Hardware Is Hard to Use and

What That Means for Real Problems

Molly Maskrey

September 2026

Personal Working Notes

Abstract

Quantum computing noise is not a single phenomenon. It is a family of distinct physical processes — each with different causes, different timescales, and different implications for algorithm design. This paper works through the major noise sources on superconducting quantum hardware (IBM transmon QPUs specifically), organized along two dimensions: noise that grows with circuit depth (gate count and time) and noise that grows with qubit count (crosstalk, frequency crowding, connectivity constraints). It closes with the honest state of the field: the gap between near-term noisy hardware and fault-tolerant quantum computing, what error mitigation can and cannot do, and what this means specifically for QAOA-based applications like MA enrollment cap detection and PBM rebate optimization.

Contents

1 Two Dimensions of Noise **3**

2 Noise Dimension 1: Circuit Depth **3**
- 2.1 T1: Energy Relaxation . . . 3
- 2.2 T2: Phase Coherence . . . 3
- 2.3 Gate Errors . . . 4
- 2.4 The Critical Budget: Gates vs. Coherence Time . . . 5

3 Noise Dimension 2: Qubit Count **5**
- 3.1 Crosstalk . . . 5
- 3.2 Frequency Crowding . . . 6
- 3.3 Connectivity Constraints and SWAP Overhead . . . 6
- 3.4 The QUBO Embedding Problem . . . 7
- 3.5 Measurement Crosstalk . . . 7

4 Error Mitigation vs. Error Correction **8**
- 4.1 Error Mitigation . . . 8
- 4.2 Error Correction . . . 8

5 What This Means for QAOA **9**
- 5.1 The Depth–Quality Tradeoff . . . 9
- 5.2 The Honest Assessment for Healthcare Applications . . . 9
- 5.3 Practical Mitigations for QAOA . . . 10

6 The Current State of the Field **10**
- 6.1 NISQ: The Era We Are In . . . 10
- 6.2 The Physical Qubit Quality Roadmap . . . 10
- 6.3 What Improves in the Meantime . . . 11

Two Dimensions of Noise

Quantum hardware noise has two largely independent axes. Understanding them separately is essential before addressing mitigations.

Dimension	Primary cause	Grows with
Circuit depth	Gate errors + decoherence	More gates, longer runtime
Qubit count	Crosstalk + connectivity	More qubits on chip

A shallow circuit on many qubits has a different noise profile from a deep circuit on few qubits. QAOA sits squarely in the intersection: it uses many qubits (one per decision variable) and its quality improves with depth p — creating an unavoidable tension between algorithmic improvement and hardware noise.

▶ **For Management:** Think of it as two separate enemies. The first enemy is time: the longer the circuit runs, the more the quantum state degrades. The second enemy is crowding: the more qubits you use simultaneously, the more they interfere with each other uninvited. Good quantum algorithm design fights both enemies at once, usually by accepting a worse approximation quality in exchange for a shorter, sparser circuit that the hardware can actually execute reliably.

Noise Dimension 1: Circuit Depth

T1: Energy Relaxation

Every qubit in superposition is in an excited state relative to its ground state. Left alone, it will eventually decay back to $|0\rangle$ by releasing a photon into the environment. The characteristic timescale for this decay is T_1, the energy relaxation time.

$$P(|1\rangle \to |0\rangle \text{ by time } t) = 1 - e^{-t/T_1} \tag{1}$$

On current IBM transmon hardware T_1 is typically 100–500 microseconds. A gate that takes 35 nanoseconds consumes roughly $35/T_1 \approx 0.007\%$ of the T_1 budget per gate. That sounds small until you consider that a meaningful QAOA circuit might contain hundreds of gates.

▶ **Hardware Reality:** T_1 decay is irreversible without error correction. Once the qubit has relaxed to $|0\rangle$, the quantum information in that qubit is gone. No amount of classical post-processing can recover it.

T2: Phase Coherence

T_2 is the phase coherence time — how long the relative phase between $|0\rangle$ and $|1\rangle$ remains stable. This is distinct from T_1. A qubit can still be in the right energy state (not yet relaxed) while its phase has already randomized.

$$T_2 \leq 2T_1 \tag{2}$$

This inequality is fundamental. Phase coherence is always lost at least as fast as energy relaxation, and usually faster. The two contributions to T_2 dephasing are:

- T_1 **relaxation itself** — energy relaxation destroys phase coherence as a side effect
- **Pure dephasing** T_ϕ — environmental fluctuations (charge noise, flux noise, two-level system defects in the Josephson junction) randomly perturb the qubit's resonant frequency, causing the phase to drift without energy exchange

The relationship is:

$$\frac{1}{T_2} = \frac{1}{2T_1} + \frac{1}{T_\phi} \tag{3}$$

T_2 on current IBM hardware is typically 50–300 microseconds, often significantly shorter than T_1 due to pure dephasing.

▶ **Hardware Reality:** T_2 is the computation enemy. As established in the companion paper on qubit state equations, the phase ϕ is the computational resource in QAOA — it is what drives interference. When T_2 expires, the phase randomizes and the interference pattern collapses. A circuit that runs longer than T_2 returns noise regardless of how well the gates were calibrated.

▶ **For Management:** T_1 is the qubit forgetting which state it is in. T_2 is the qubit forgetting the timing of its wave. In the light source analogy: T_1 is the light source turning off unexpectedly. T_2 is the light source staying on but losing its phase relationship with the other sources — so instead of a deliberate interference pattern on the wall, you get random flickering. T_2 kills the computation even when the qubit is still physically alive.

Gate Errors

Every physical gate introduces a small but non-zero error. On IBM hardware the approximate gate fidelities are:

Gate	Error rate	Notes
$R_Z(\lambda)$	≈ 0	Virtual — no pulse, no error
SX ($\sqrt{X}$)	$\sim 0.1\%$	Single-qubit physical gate
CNOT (CX)	~ 0.5–1%	Two-qubit gate, dominant error source
Readout	~ 1–2%	Measurement itself introduces error

Gate errors compound multiplicatively. A circuit with k CNOT gates has fidelity approximately:

$$\mathcal{F}_{\text{circuit}} \approx (1 - \epsilon_{\text{CX}})^k \tag{4}$$

where $\epsilon_{\text{CX}} \approx 0.005$–$0.01$. At $k = 100$ CNOT gates:

$$\mathcal{F} \approx (0.995)^{100} \approx 0.61 \quad \text{to} \quad (0.99)^{100} \approx 0.37$$

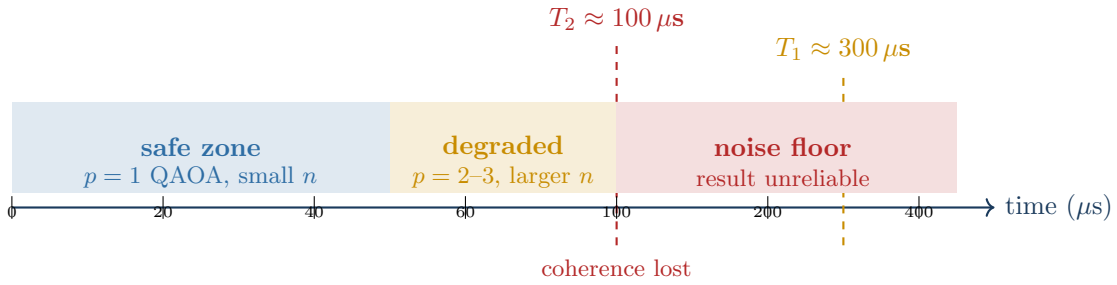
You have lost between 39% and 63% of your circuit fidelity from gate errors alone, before decoherence has had any additional effect.

The Critical Budget: Gates vs. Coherence Time

The total circuit execution time is approximately:

$$t_{\text{circuit}} \approx n_{\text{SX}} \cdot t_{\text{SX}} + n_{\text{CX}} \cdot t_{\text{CX}} + t_{\text{readout}} \tag{5}$$

where $t_{\text{SX}} \approx 35\,\text{ns}$, $t_{\text{CX}} \approx 300\,\text{ns}$, and $t_{\text{readout}} \approx 1$–$10\,\mu\text{s}$. This must be kept well below T_2.



▶ **Hardware Reality:** The readout step alone — measuring all qubits at the end — takes 1–10 microseconds, a significant fraction of T_2. On a noisy device the measurement itself is often the largest single source of error in a shallow circuit.

Noise Dimension 2: Qubit Count

Crosstalk

Qubits on an IBM chip share a common microwave environment. A pulse applied to qubit A to implement a gate is felt, weakly, by neighboring qubits B and C. This unintended coupling is crosstalk.

The crosstalk problem compounds with qubit count in two ways:

1. **More active qubits** means more pulses executing simultaneously (parallelism is necessary to keep circuit time short), and more pulses means more leakage into neighbors.
2. **More neighbors** means more paths through which crosstalk can propagate. On a chip with 100 qubits, perturbations can cascade through the connectivity graph in ways that are difficult to characterize.

IBM partially mitigates crosstalk through **frequency allocation**: neighboring qubits are tuned to different resonant frequencies so that pulses intended for one qubit are off-resonance for its neighbors and therefore less likely to cause transitions. This works well when the frequency separation is large. It becomes harder to maintain as qubit count grows.

Frequency Crowding

Each transmon qubit requires a dedicated resonant frequency in the 4–8 GHz band. The usable frequency space is not unlimited. As more qubits are added to a chip, two problems emerge:

- **Frequency collisions**: qubits whose resonant frequencies are too close experience stronger unintended coupling, effectively becoming permanently partially entangled whether you want them to be or not.
- **Anharmonicity constraints**: transmon qubits are not perfect two-level systems. They have higher energy levels ($|2\rangle$, $|3\rangle$, ...) that must be kept well-separated from the operating frequency to avoid leakage out of the computational subspace. This further constrains usable frequency space.

Frequency crowding is one of the fundamental reasons that simply adding more qubits to a chip does not scale linearly in usable computational capacity.

> ▶ **For Management:** Imagine assigning radio frequencies to many stations on the same tower. Each station needs its own clear channel. As you add more stations, the channels get closer together, the stations start bleeding into each other, and eventually the whole system becomes unintelligible noise. Qubit frequency crowding is the same problem at 5 GHz and 15 millikelvin.

Connectivity Constraints and SWAP Overhead

IBM's quantum chips are not fully connected. Each qubit is physically coupled to only 2–5 neighbors, depending on the chip architecture. The connectivity graph is sparse by design — more connections mean more crosstalk.

This creates a fundamental problem for algorithms whose structure requires interactions between non-adjacent qubits. If your QAOA circuit needs to entangle qubit i and qubit j but they are not physically adjacent, Qiskit must route the quantum state through intermediate qubits using SWAP gates.

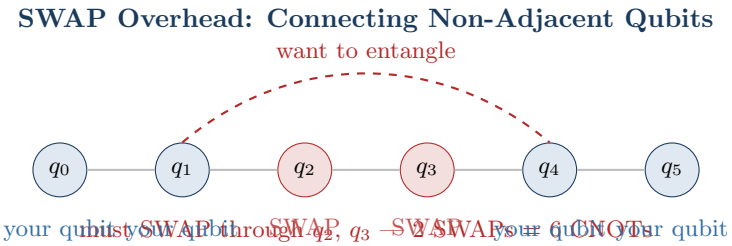
A SWAP gate is implemented as three CNOT gates:

$$\mathrm{SWAP}(i,j) = \mathrm{CX}(i,j) \cdot \mathrm{CX}(j,i) \cdot \mathrm{CX}(i,j) \tag{6}$$

Each SWAP therefore costs:

- 3 CNOT gate errors ($\approx$ 1.5–3% combined error)
- $\approx$ 900 ns of circuit time
- Decoherence on both qubits for that duration

A circuit that requires s SWAP operations has its fidelity reduced by approximately $(1 - 0.01)^{3s}$ from SWAP overhead alone, before any of the circuit's intended gates are counted.



The QUBO Embedding Problem

For QAOA specifically, the QUBO connectivity directly determines the SWAP overhead. The QUBO matrix Q has non-zero off-diagonal terms Q_{ij} wherever two decision variables are coupled. Each non-zero Q_{ij} requires an entangling gate between qubits i and j. If those qubits are not physically adjacent, SWAP gates are required.

QUBO structure	Hardware implication
Sparse (few off-diagonal terms)	Low SWAP overhead, maps well to hardware
Dense (many off-diagonal terms)	High SWAP overhead, rapidly becomes intractable
Structured (e.g. organizational hierarchy)	Can be mapped to exploit hardware topology
Random dense	Nearly impossible to run cleanly on current hardware

For the MA enrollment cap detection problem: the diagonal terms (individual cap evidence scores) require no entanglement — only R_Z gates, which are virtual and free. The off-diagonal terms (organizational coupling between plans sharing a parent organization) require CNOT gates. If the QUBO is structured so that organizationally linked plans can be assigned to physically adjacent qubits, SWAP overhead is minimized. This is a non-trivial layout optimization problem that Qiskit's transpiler attempts automatically but does not always solve optimally.

▶ **Hardware Reality:** A densely connected QUBO on a sparsely connected chip can generate more SWAP overhead than the original circuit gates. It is entirely possible to design a theoretically correct QAOA circuit that is practically unrunnable on available hardware because the embedding overhead pushes the circuit time beyond T_2.

Measurement Crosstalk

Simultaneous readout of multiple qubits introduces correlated errors. The dispersive readout pulse for qubit A travels through the same microwave environment as qubit B's readout resonator. On current IBM hardware, measurement error rates increase when many qubits are read simultaneously compared to reading them one at a time.

This matters for QAOA because the final measurement step reads all n qubits simultaneously to extract the solution bitstring. On a 50-qubit circuit, simultaneous readout of 50 qubits accumulates correlated measurement errors that are difficult to characterize and correct.

Error Mitigation vs. Error Correction

These are fundamentally different approaches and the distinction matters for setting honest expectations about near-term hardware.

Error Mitigation

Error mitigation techniques attempt to reduce the effect of noise on measurement outcomes without actually preventing or correcting the underlying errors. They work classically, in post-processing, by running the noisy circuit multiple times under different conditions and combining the results.

Common mitigation techniques:

Zero-Noise Extrapolation (ZNE): Run the circuit at artificially amplified noise levels (by stretching gate pulses) and extrapolate back to the zero-noise limit. Requires multiple circuit executions.

Probabilistic Error Cancellation (PEC): Decompose noisy gates into combinations of ideal gates and random Pauli errors, then average over many circuit instances. Exponentially expensive in shot count.

Measurement Error Mitigation: Characterize the readout error matrix (probability of measuring $|0\rangle$ given the qubit is in $|1\rangle$ and vice versa) and invert it classically on the measurement statistics. Cheap and effective for measurement errors specifically.

What mitigation can do: reduce the effective noise level, improve expectation value estimates, make shallow circuits more reliable.

What mitigation cannot do: recover information lost to decoherence, fix errors in deep circuits, substitute for error correction.

Error Correction

Quantum error correction (QEC) physically encodes one *logical* qubit in many *physical* qubits, using the redundancy to detect and correct errors without measuring the logical state directly.

The most promising QEC scheme for superconducting hardware is the **surface code**. A single logical qubit encoded in a surface code requires approximately:

$$n_{\text{physical}} \approx 1{,}000 \text{ physical qubits per logical qubit} \tag{7}$$

at current physical gate error rates ($\sim 0.1\%$ for single-qubit, $\sim 1\%$ for two-qubit gates). To run a fault-tolerant QAOA circuit with 50 logical qubits would require approximately 50,000 physical qubits operating below the surface code error threshold.

Current IBM hardware: 100–1,000+ physical qubits, all noisy. Required for fault tolerance: millions of physical qubits below threshold.

▶ **Hardware Reality:** The gap between current noisy intermediate-scale quantum (NISQ)

hardware and fault-tolerant quantum computing is not a software problem. It is a physics and engineering problem that will take years to decades to close. All near-term quantum algorithms — including QAOA — operate in the NISQ regime and are subject to all the noise sources described in this document.

▶ **For Management:** Error correction is like building a RAID array: you use many disks to protect against any one disk failing, with enough redundancy that the system keeps working even when individual components are unreliable. Quantum error correction does the same thing with qubits. The problem is that the overhead is enormous — you need roughly 1,000 noisy physical qubits to make one reliable logical qubit. We do not yet have enough physical qubits, and the ones we have are not yet reliable enough, to build a useful fault-tolerant quantum computer. Everything we can do today is on noisy hardware without error correction.

What This Means for QAOA

The Depth–Quality Tradeoff

QAOA's approximation quality improves monotonically with the number of layers p. In the limit $p \to \infty$, QAOA converges to the exact solution. In practice:

p	**Parameters**	**Hardware reality**
1	2 (γ, β)	Feasible on current hardware, modest quality
2	4	Marginal — noise starts to dominate for large n
3	6	Difficult — circuit time approaches T_2 for many problems
> 3	$2p$	Generally unreliable on current NISQ hardware

The crossover point — where adding a QAOA layer makes the answer *worse* due to accumulated noise rather than better due to improved approximation — depends on the specific hardware, the qubit count, the QUBO connectivity, and the day-to-day calibration state of the device.

The Honest Assessment for Healthcare Applications

For the MA enrollment cap detection problem and the PBM rebate optimization problem, the honest hardware assessment at the time of writing is:

- **Small instances** (10–20 qubits, sparse QUBO, $p = 1$): runnable on IBM hardware today with error mitigation. Results are probabilistic but meaningfully better than random. Useful as a proof of concept.
- **Medium instances** (50–100 qubits, moderate QUBO density, $p = 1$–2): at the edge of what current hardware can do reliably. Results are noisy and require careful error mitigation and many repeated shots to extract signal.
- **Full-scale instances** (hundreds of plan-county variables, dense organizational coupling, $p \geq 3$): not reliably executable on current NISQ hardware. Classical heuristics (simulated annealing, tabu search) will outperform noisy QAOA at this scale.

- **Fault-tolerant instances**: years to decades away, depending on progress in physical qubit quality and error correction overhead reduction.

> ▶ **Hardware Reality:** This is not a reason to abandon QAOA for these problems. It is a reason to be honest about the timeline and to start with small validated instances that demonstrate the framework works, then scale as hardware improves. The QUBO formulations, the RBM training pipeline, and the classical optimization loop are hardware-agnostic. They will work better on better hardware. The investment in the framework is not wasted by current hardware limitations.

Practical Mitigations for QAOA

Given the noise constraints, several practical strategies improve QAOA performance on near-term hardware:

Keep p small. Start at $p = 1$ and only increase if the noise budget allows. Accept a worse approximation ratio in exchange for a circuit the hardware can actually execute.

Exploit QUBO structure. Design the QUBO so that coupled variables can be assigned to physically adjacent qubits. For the MA cap detection problem this means ordering plan-county qubits so that organizationally linked plans are adjacent in the hardware connectivity graph.

Use error mitigation. Measurement error mitigation is cheap and effective. Zero-noise extrapolation improves expectation value estimates at the cost of more circuit executions.

Warm-start the optimizer. Good initial γ, β values reduce the number of circuit evaluations the classical optimizer needs, reducing total shot count and therefore cost.

Use the simulator first. Qiskit Aer's noise models can simulate realistic IBM hardware noise. Validate that the circuit produces meaningful results on a noise model before spending shots on real hardware.

Choose the right device. IBM publishes daily calibration data for all its devices. Run on the device with the best current gate fidelities and T_2 times. These vary significantly day to day.

The Current State of the Field

NISQ: The Era We Are In

The term **NISQ** (Noisy Intermediate-Scale Quantum) was coined by John Preskill in 2018 to describe the quantum hardware available in the near term: devices with 50–1,000+ qubits that are too noisy for error correction but too large to simulate classically.

NISQ devices can demonstrate quantum phenomena and run shallow circuits with meaningful results. They cannot yet demonstrate fault-tolerant quantum advantage on problems of practical scale.

The Physical Qubit Quality Roadmap

The surface code error threshold — the gate fidelity below which errors can be corrected faster than they accumulate — is approximately 1% for two-qubit gates. Current IBM hardware is at or

just below this threshold for the best devices, on the best days.

Getting reliably below the threshold and keeping thousands of qubits there simultaneously is the central engineering challenge of the next decade. IBM's roadmap targets fault-tolerant quantum computing in the 2030s.

What Improves in the Meantime

Even without fault tolerance, hardware improvements directly benefit QAOA:

- Higher T_1 and T_2 allow deeper circuits
- Lower CNOT error rates allow denser QUBO coupling
- Better connectivity graphs reduce SWAP overhead
- Larger qubit counts allow larger problem instances
- Better calibration routines reduce day-to-day variability

The framework developed for healthcare quantum optimization applications — the QUBO formulations, the hybrid classical-quantum training pipeline, the RBM-informed cost Hamiltonians — is ready to take advantage of hardware improvements as they arrive. The algorithms do not need to change. The hardware just needs to get better.

Summary

Noise source	**Dimension**	**Primary mitigation**
T_1 relaxation	Depth	Keep circuits short
T_2 dephasing	Depth	Keep circuits short, virtual R_Z
Gate errors (SX)	Depth	Minimize single-qubit gate count
Gate errors (CNOT)	Depth	Minimize two-qubit gate count
Crosstalk	Width	Frequency allocation, pulse shaping
Frequency crowding	Width	Chip architecture design
SWAP overhead	Width	QUBO structure, layout optimization
Measurement crosstalk	Width	Sequential readout, calibration
Error mitigation	Near-term	Reduce effective noise, not eliminate
Error correction	Long-term	$\sim$1,000 physical per logical qubit

Personal Working Notes — September 2026